

MCP-Focus: Leveraging Function-Oriented Document Enhancement for MCP Server Retrieval

Wenchun Jing
Institute for Artificial Intelligence,
Peking University
Beijing, China
jingwenchun@stu.pku.edu.cn

Haiyang Shen
Institute for Artificial Intelligence,
Peking University
Beijing, China
hyshen@stu.pku.edu.cn

Haoran Wang
Institute for Artificial Intelligence,
Peking University
Beijing, China
wanghaoran1010@stu.pku.edu.cn

Qi Liu
Gaoling School of Artificial
Intelligence, Renmin University of
China
Beijing, China
liuqi_67@ruc.edu.cn

Ningyuan Li
College of Computer Science, Beijing
University of Technology
Beijing, China
ningyu@emails.bjut.edu.cn

Chaoran Luo
National Key Laboratory of Dataspace
Technology and System
Beijing, China
luochaoran@pku.edu.cn

Ning Zhang
School of Computer Science and
Technology, Xidian University
Xi'an, Shaanxi, China
zhangning7730@xidian.edu.cn

Yun Ma*
Institute for Artificial Intelligence,
Peking University
Beijing, China
mayun@pku.edu.cn

Abstract

Model Context Protocol (MCP) has emerged as a practical standard for connecting LLM-based agents with external tools and services through MCP servers. Driven by the open-source community, the MCP ecosystem is rapidly expanding, resulting in a large and growing collection of third-party MCP servers. Accurately selecting MCP servers that satisfy functional requirements from many candidates, therefore, becomes an increasingly important problem. However, MCP server documents are often unstructured and exhibit ambiguous function semantics, making it difficult to align user requirements with server capabilities during retrieval. To address this issue, we propose MCP-Focus, a function-oriented document enhancement framework that produces retrieval-ready MCP server documentation via a multi-stage agentic pipeline for white-box code analysis and document generation. Specifically, MCP-Focus first extracts a comprehensive tool inventory with metadata, then refines tool-level descriptions grounded in each extracted tool's implementation, and finally aggregates the refined tool descriptions into a structured server-level overview as the retrieval document. To better evaluate MCP server retrieval, we construct a benchmark comprising 3k+ open-source MCP servers and human-guided queries that vary in semantic ambiguity, input-output specificity, and the number of involved function points. Experiments across multiple dense retrievers show that fine-tuning with MCP-Focus-enhanced documents consistently improves retrieval effectiveness

over baseline document methods on multiple benchmarks. Code and data: <https://github.com/JingWC/MCP-Focus>.

CCS Concepts

• **Information systems** → **Document representation**; • **Computing methodologies** → **Information extraction**.

Keywords

Model Context Protocol, API Document Generation, Tool Retrieval

ACM Reference Format:

Wenchun Jing, Haiyang Shen, Haoran Wang, Qi Liu, Ningyuan Li, Chaoran Luo, Ning Zhang, and Yun Ma. 2026. MCP-Focus: Leveraging Function-Oriented Document Enhancement for MCP Server Retrieval. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26)*, July 20–24, 2026, Melbourne, VIC, Australia. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3805712.3809582>

1 Introduction

Large language models (LLMs) have reshaped artificial intelligence by moving beyond static text generation toward agentic workflows that interact with external environments [1, 12, 24]. In this ecosystem, the Model Context Protocol (MCP) [5, 11, 27] has become a commonly used interface for connecting LLMs to data and tools. By providing a standardized interface, MCP decouples the model from specific tool implementations, allowing developers to build modular, interoperable agents. In such agentic systems, in practice, selecting MCP servers that satisfy a user's functional requirements is essential for building scalable, general-purpose autonomous agents [15, 26, 30]. However, as the open-source community embraces MCP, the number of available MCP servers has grown rapidly, creating a large repository of capabilities ranging

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGIR '26, Melbourne, VIC, Australia*.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2599-9/2026/07

<https://doi.org/10.1145/3805712.3809582>

Table 1: Document quality statistics for 1,247 popular MCP servers collected from Glama.ai. We employ an *LLM-as-a-judge* to assess both structural integrity (covering server and tool descriptions, as well as tool input/output information) and semantic clarity of server and tool descriptions. Manual verification on 100 sampled servers shows over 80% agreement with the automated ratings. Full scoring details are provided in our code repository.

Issue \ Score		0	1	2	3
README	Exist	2.41%	97.59%	-	-
Structural Integrity	Server desc.	5.05%	94.95%	-	-
	Tool desc.	5.93%	94.07%	-	-
	Tool input	18.68%	56.05%	25.26%	-
	Tool output	85.49%	8.66%	5.85%	-
Semantic Clarity	Server desc.	2.41%	2.81%	72.65%	22.13%
	Tool desc.	2.41%	1.92%	64.15%	31.52%

from web search to local file manipulation [8, 9, 18]. As a result, accurately retrieving the most relevant MCP server for a specific user request has become a critical bottleneck.

However, accurately identifying the MCP server remains challenging due to practical issues that are underexplored in existing methods. Prior work on tool and API retrieval mainly emphasizes algorithmic advances, such as improved loss functions [22, 39], more effective negative sampling [3], or retrievers that support larger context windows [40]. A few data-centric efforts, such as ScaleMCP [17], aim to improve retrieval by constructing better datasets, but they typically assume that tools come with complete and well-structured documents, which is an assumption that rarely holds in the open-source MCP ecosystem. In practice, as shown in Table 1, the native document for MCP servers is typically limited to project README files, which lack consistent standards. As a result, README descriptions of individual functions often miss tool information or critical retrieval cues (e.g., parameters and expected outputs), and both server-level and tool-level descriptions frequently exhibit limited semantic clarity. This mismatch between increasingly advanced retrievers and documents of limited quality ultimately constrains the practical effectiveness of MCP retrieval.

To bridge this gap, we propose MCP-Focus, a framework that improves MCP retrieval by enhancing MCP documents rather than optimizing the retriever. Our core idea is to generate retrieval-oriented documents that provide accurate and comprehensive descriptions of both server-level capabilities and individual tools by white-box analysis of MCP server implementations, rather than relying on README files for indexing. However, existing general-purpose repository-level document generation [19] lacks the precision to distinguish between internal utility functions and exposed MCP tools. To address this challenge, we adopt an MCP-specific, bottom-up strategy that first isolates fine-grained semantics of externally exposed MCP tools and then aggregates them into coherent server-level semantics.

Concretely, MCP-Focus is realized as a multi-stage, LLM-driven agentic pipeline: the *Tool Extractor* navigates a full MCP server repository, where tools are distributed across multiple files and modules, to identify valid MCP tool entry points and extract interface metadata (e.g., parameter and return schemas) while mapping each tool to its code location; the *Tool Document Refiner* uses the extracted mappings to locate each tool’s implementation and extends beyond surface-level signatures by iteratively analyzing localized code to generate tool documents with fine-grained functional descriptions; and the *Server Document Refiner* synthesizes tool-level evidence into a server-level overview. We then build retrieval chunks by concatenating the overview with each tool document, yielding chunks that preserve global context and local tool details.

In addition to the defects in the document, establishing a rigorous evaluation for MCP retrieval is challenging: a robust benchmark must cover a broad range of retrieval scenarios, yet existing datasets remain insufficient. First, *limited server heterogeneity*: current benchmarks do not match the scale and diversity of the open-source ecosystem. For instance, ScaleMCP [17] is finance-oriented, LiveMCPBench [21] focuses on daily tasks across six domains, and MCPMark [36] and MCP-Bench [35] rely on relatively small, manually curated server sets. Second, *limited query complexity*: queries in MCP-RADAR [7], MCP-Universe [20], and MCP-Flow [34] are often explicit execution instructions, under-representing the ambiguity and constraint-heavy requirements typical of real search.

To address these limitations, we construct a new benchmark that reflects the diversity of the MCP ecosystem. It comprises 3,763 real-world open-source servers collected from diverse repositories, and we design a query generation framework spanning single-function lookups to complex scenario tasks. Our query construction follows the workflow of MCP Eval [16]: we generate queries from tool schemas validated by Glama¹ and combine LLM generation with human review and verification to ensure realism and consistency in a retrieval-focused setting. We evaluate on our benchmark as well as two public benchmarks, MCP-Bench [35] and MCP-Universe [20], to assess generalization. We further conduct ablation studies on key components and analyze performance on complex multi-server queries. Results show that MCP-Focus consistently outperforms all baselines across all metrics.

In summary, our contributions are as follows:

- We present a document-centric perspective for MCP retrieval that prioritizes document quality over retriever complexity; to our knowledge, we propose the first function-oriented document enhancement framework for the MCP ecosystem.
- We develop a multi-stage agentic pipeline that leverages white-box code analysis to generate a structurally complete and semantically accurate document from raw MCP server source code.
- We construct a large-scale, heterogeneous benchmark for MCP server retrieval, featuring 3,763 servers and a diverse set of queries with varying complexity.
- We conduct extensive experiments showing that our document enhancement strategy improves retrieval performance across multiple state-of-the-art retrievers and public benchmarks.

¹<https://glama.ai/mcp>

2 Related Work

2.1 Dense Retrieval Optimization

Research in dense retrieval generally aims to improve how models map queries and documents into a shared vector space. We can categorize recent advancements into three main directions: algorithmic optimization, model scaling, and data-centric approaches. **Algorithmic approaches** focus on refining the learning objective. Methods like ANCE [38] introduce approximate nearest neighbor negative sampling to challenge the model during training. TAS-B [10] utilizes topic-aware sampling and balanced optimization, while other works [39] employ contrastive loss functions to improve embedding quality. **Scaling approaches** rely on increasing model capacity and batch size. Models such as GTR [23], NV-Embed [14], and E5 [33] demonstrate that larger backbone models and massive batch training significantly enhance semantic understanding. **Data-centric approaches** focus on improving the training data. However, most existing work here focuses on *query generation*. Techniques like PromptGator [4], InPars [2], and GPL [32] use LLMs to synthesize queries for existing documents to augment training sets.

Our work is most closely related to data-centric methods but targets a different axis of improvement: *document* quality. This focus is motivated by pervasive document deficiencies in open-source MCP servers. As shown in Table 1, tool usage is frequently underspecified: tool inputs are missing for roughly 1/5 of servers, and tool outputs are absent for most (over 80%). In addition, only a minority of server/tool descriptions reach the highest semantic-clarity score (about 20-30%). We argue that such incomplete and ambiguous documents can become a bottleneck for MCP server retrieval. MCP-Focus mitigates this issue by analyzing repository code and generating function-oriented documents, yielding more informative representations for downstream retrieval.

2.2 Tool and API Retrieval

Tool and API retrieval has gained traction with autonomous agents and is often formulated as matching instructions to standardized API specifications. Gorilla [25], ToolLLM [26], and AutoTools [29] focus on fine-tuning models to select tools from well-formed JSON or OpenAPI definitions, implicitly assuming that the specification is complete and accurate. However, this assumption is frequently violated in the open-source MCP ecosystem, where server documentation is often missing or outdated, making metadata-only retrieval unreliable.

MCP servers also exhibit higher structural complexity than conventional REST APIs: beyond a flat endpoint list, they contain local control flow, internal dependencies, and hierarchical tool relationships [21]. Accordingly, black-box retrieval that treats a server as an opaque text block is limited. In contrast, our approach adopts a white-box perspective by performing agentic repository analysis to identify exposed tools and generate behavior-grounded descriptions from code.

2.3 MCP Evaluation Benchmarks

Robust benchmarks are necessary to quantify progress in MCP server retrieval, yet existing datasets do not fully reflect the scale and diversity of the open-source MCP ecosystem. We observe two

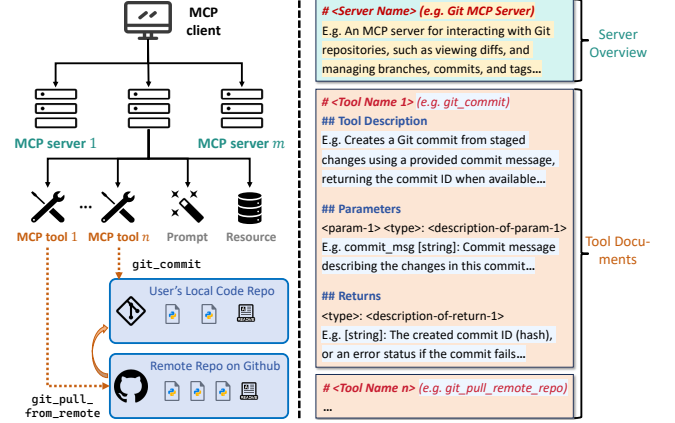


Figure 1: MCP architecture and function-oriented document. The left panel illustrates the MCP architecture, including an MCP client, MCP servers, and their exposed tools, resources, and prompts; the right panel shows function-oriented MCP document representation, consisting of a server overview and tool documents. Using a Git MCP server as an example.

recurring limitations. First, several benchmarks, including MCP-RADAR [7], MCP-Universe [20], and MCP-Flow [34], primarily evaluate the *execution* stage, i.e., whether an agent can invoke tools in a known environment, while simplifying or de-emphasizing retrieval. Second, benchmarks that incorporate retrieval are often constrained in server heterogeneity or query complexity. ScaleMCP [17] focuses on a single domain (finance) with relatively homogeneous queries, LiveMCPBench [21] centers on daily tasks across six domains, and MCPMark [36] and MCP-Bench [35] rely on small, manually curated server lists. Consequently, these benchmarks under-represent the ambiguity and constraint-heavy requirements typical of real-world search and only partially capture the “in-the-wild” characteristics of open-source MCP servers.

To address these gaps, we construct a benchmark explicitly targeted at MCP server retrieval under realistic server heterogeneity and query complexity. Compared with prior datasets, our benchmark includes 3,763 diverse open-source servers and organizes queries into multiple types spanning difficulty levels, from explicit single-tool lookups to complex scenario tasks. This design aims to provide a more rigorous testbed for retrieval in large-scale, real-world MCP settings.

3 Background

This section provides an overview of the Model Context Protocol (MCP) architecture and describes how we represent its components in a structured format suitable for MCP retrieval.

3.1 Model Context Protocol (MCP)

The Model Context Protocol (MCP) has emerged as a standard for connecting LLMs to external data and tools. As illustrated in Figure 1 (left), an MCP architecture comprises a host application (MCP client) and independent services (MCP servers). Unlike traditional RESTful APIs exposed through remote endpoints, an MCP server

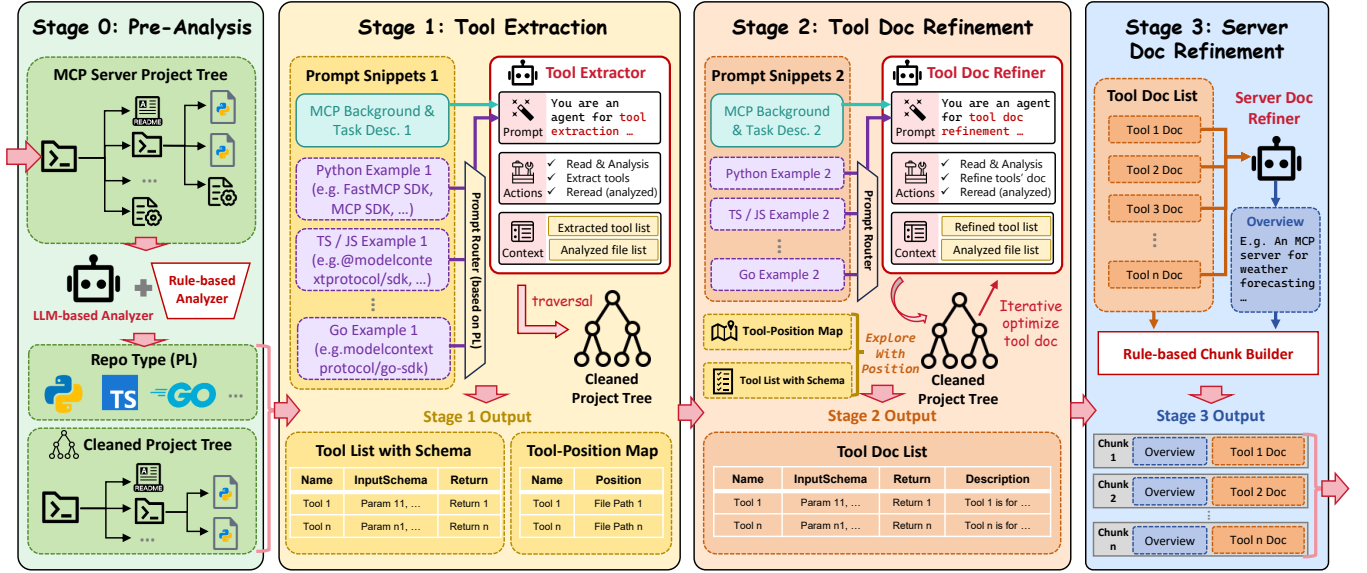


Figure 2: MCP-Focus Framework. A four-stage agentic pipeline converts an MCP repository into function-oriented documents for retrieval: (1) pre-analysis: cleans the project and reduces noise, (2) tool extraction: identifies exposed MCP tools and their positions, (3) tool document refinement: derives tool semantics from localized code context, and (4) server document refinement: aggregates tool-level evidence into a coherent server overview.

is often implemented as a lightweight local process. It provides three primary capabilities, referred to as primitives: **Resources**, which represent data that can be read by the model (e.g., log files, database records, or system state); **Prompts**, which are predefined templates that facilitate effective interaction with the server; and **Tools**, which are executable functions that enable the model to perform actions (e.g., executing a script or modifying a file).

For autonomous agents, **Tools** are particularly important because they enable LLMs to interact with external environments. However, in the open-source ecosystem, tool implementations are frequently embedded within complex codebases and lack standardized documentation. This structural complexity makes it challenging for existing retrieval systems to identify the appropriate MCP servers.

3.2 Function-Oriented Document Representation

To enable accurate retrieval of MCP servers, we linearize the hierarchical repository structure into a semantically dense textual representation. As shown in Figure 1 (right), we do not index an entire repository as a single retrieval unit. Instead, we adopt a *function-oriented* representation that jointly captures the server-level context and the tool-level functional details.

Specifically, each retrieval unit is organized into two components. **Server Overview** ($Doc_{overview}$): This component summarizes the project’s high-level purpose, including the server name and a global description derived from the repository’s intent, thereby providing domain context for retrieval (e.g., database management or web scraping). **Tool Documents** (Doc_{tool}): Following the overview, we represent individual tools at a fine granularity. Each tool document

Doc_{tool_i} contains the tool name and a generated semantic description ($Str_{tool_desc_i}$) that specifies *what* the tool does, together with a detailed interface specification of parameters (Doc_{paras_i}) and return values ($Doc_{returns_i}$). These interface descriptions provide discriminative signals for precise queries, such as those involving particular input formats (e.g., JSON).

For servers that expose many tools, we partition the documentation into indexable chunks. Each chunk concatenates the shared $Doc_{overview}$ with a single Doc_{tool_i} . This design enables tool-level matching while preserving essential server-level context.

4 Method

In this section, we introduce MCP-Focus, a document enhancement framework for MCP server retrieval, addressing the limited coverage and inconsistency of README-based descriptions. We implement MCP-Focus as a bottom-up, agentic pipeline that decomposes each server into externally exposed MCP tools, refines fine-grained tool semantics via localized code analysis, and then aggregates tool-level evidence into a coherent server-level document.

As illustrated in Figure 2, our pipeline proceeds in 4 stages: (1) **Pre-Analysis** to reduce noise, (2) **Tool Extraction** to locate and standardize tools, (3) **Tool Document Refinement** to generate tool documents grounded in the code implementation, and (4) **Server Document Refinement** to generate the server document. In each stage, we design a dedicated LLM-based agent to accomplish the corresponding task.

4.1 Stage 0: Repository Pre-Analysis

Input and Output Input: Raw repository with file hierarchy. Output: Detected language and cleaned project tree.

Real-world MCP repositories are often cluttered, frequently containing build artifacts, test suites, and vendored dependencies that are irrelevant to the server’s external capabilities. Feeding this structural noise into an LLM-based agent dilutes the context and leads to the "lost-in-the-middle" phenomenon. To mitigate this, we employ a lightweight Pre-Analysis stage that infers the implementation context and purges irrelevant data.

Given a repository, we denote its raw project tree as $Tree(Repo)$, i.e., the hierarchy of paths in the repository. An *LLM-based Analyzer* predicts a dominant implementation language $L(Repo) \in \mathcal{L}$, where $\mathcal{L}$ denotes the set of MCP-supported languages. Rather than relying solely on GitHub repository metadata, which can be misleading when extra code is present, the analyzer infers the language from structural cues in $Tree(Repo)$ (e.g., the presence of a `server.py` file strongly suggests a Python-based MCP server).

Conditioned on the detected language, a rule-based analyzer then performs repository cleaning to generate the optimized tree:

$$Tree_{clean}(Repo) = Clean(Tree(Repo), L(Repo)) \quad (1)$$

This operation retains only the source code files and the README (preserved as a primary semantic reference), while filtering out irrelevant directories and binary artifacts. Consequently, this stage outputs both the detected programming language and the noise-reduced project tree to maximize information density for downstream agents.

4.2 Stage 1: Tool Extraction

Input and Output Given the detected implementation language $L(Repo)$ and the cleaned project tree $Tree_{clean}(Repo)$, the extractor produces two structured artifacts. The first is a *Tool List with Schema* $S(Repo)$, defined as:

$$S(Repo) = [s_1, \dots, s_m] \quad (2)$$

where each item s_i encapsulates the tool’s interface:

$$s_i = \langle ToolName_i, InputSchema_i, Return_i \rangle \quad (3)$$

Here, $InputSchema_i = \langle name, type \rangle$ and $Return_i = \langle name, type \rangle$ (when available). The second output is a *Tool $\rightarrow$ Position Map* $\mathcal{P}(Repo)$ that links each extracted tool to its precise implementation file path and span in $Tree_{clean}(Repo)$.

The core challenge in this stage is identifying valid MCP tool entry points that are often fragmented across modules (e.g., a tool defined in one file but registered in another via a variable). A single-pass scan often fails to resolve such indirect references. To address this, the *Tool Extractor* agent employs an Ambiguity-Aware Deferred Resolution strategy. This approach prioritizes gathering explicit definitions to build a repository-wide context before iteratively resolving complex or incomplete registrations.

The extraction process, formalized in Algorithm 1, begins by initializing a queue Q with heuristic entry points (e.g., `src/server.py`) selected from candidate source files $F_{cand}(Repo)$. The agent traverses these files to identify tool patterns using language-specific idioms (e.g., `@mcp.tool` decorators). During this traversal, the agent assesses the *semantic completeness* of each detected candidate.

- **Direct Extraction:** If a tool definition is self-contained (e.g., a function with a complete signature and explicit registration), it is immediately extracted into $S(Repo)$.

- **Ambiguity Detection & Buffering:** If a registration is detected but lacks immediate definition (e.g., `server.add_tool(my_tool)` where `my_tool` is imported or dynamically defined), the agent marks it as "ambiguous." Instead of hallucinating details or failing immediately, the agent buffers this file in an *AmbiguousBuffer*. The motivation is that subsequent traversal of other files may reveal the missing definition (e.g., the source of the imported `my_tool`).

Simultaneously, the agent systematically traces import statements; if a file imports unvisited modules, the agent adds them to Q . Once the primary traversal queue Q is exhausted, the agent enters the Resolution Phase. It iteratively revisits the *AmbiguousBuffer*, utilizing the now-populated global context (known imports and definitions from the full scan) to resolve the previously unclear signatures. This process repeats until no new MCP tools are resolved or a preset retry budget is exhausted, ensuring the agent maximizes tool discovery while guaranteeing termination.

Algorithm 1 Extraction with Deferred Resolution

Require: $Tree_{clean}(Repo), L(Repo)$

```

1:  $Q \leftarrow \text{HeuristicEntryPoints}(Tree_{clean})$ 
2:  $S \leftarrow \emptyset, \mathcal{P} \leftarrow \emptyset, \text{AmbiguousBuffer} \leftarrow \emptyset$ 
3: while  $Q$  is not empty or ( $\text{AmbiguousBuffer} \neq \emptyset$  and
    $\text{Budget} > 0$ ) do
4:   if  $Q$  is empty then
5:      $Q \leftarrow \text{AmbiguousBuffer}; \text{AmbiguousBuffer} \leftarrow \emptyset$ 
6:      $\text{Budget} \leftarrow \text{Budget} - 1$ 
7:   end if
8:    $f \leftarrow Q.pop()$ 
9:    $Code \leftarrow \text{ReadFile}(f)$ 
10:   $\text{Candidates}, \text{Imports} \leftarrow \text{AgentAnalyze}(Code, L(Repo))$ 
11:  for all  $c \in \text{Candidates}$  do
12:    if  $\text{IsComplete}(c)$  then
13:       $S.append(\text{ExtractSchema}(c))$ 
14:       $\mathcal{P}[c.name] \leftarrow (f, c.span)$ 
15:    else
16:       $\text{AmbiguousBuffer.add}(f)$ 
17:    end if
18:  end for
19:  for all  $path \in \text{Imports}$  do
20:    if  $path \in Tree_{clean}$  and  $path$  not visited then
21:       $Q.push(path)$ 
22:    end if
23:  end for
24: end while
25: return  $S, \mathcal{P}$ 

```

4.3 Stage 2: Tool Document Refinement

Input and Output Formally, the inputs to the tool document refinement stage are the Tool List with Schema $S(Repo) = [s_1, \dots, s_m]$ and the Tool-Position Map $\mathcal{P}(Repo)$ produced by Stage 1. The output is a *tool document list*:

$$\mathcal{D}(Repo) = [d_1, \dots, d_m] \quad (4)$$

where each item d_i extends the original schema with textual descriptions:

$$d_i = \langle ToolName_i, ToolDesc_i, InputSchema'_i, Return'_i \rangle \quad (5)$$

In this structure, $ToolDesc_i$ summarizes the tool's intent, while $InputSchema'_i$ and $Return'_i$ augment each field into a tuple of $\langle name, type, desc \rangle$.

Function signatures specify invocation but omit usage semantics; for example, a parameter named `threshold` is ambiguous without knowing if it acts as a timeout limit or a confidence cutoff. Moreover, treating tools in isolation often leads to generic descriptions that fail to distinguish between functionally similar tools (e.g., `quick_search` vs. `deep_search`). To address these challenges, the *Tool Document Refiner* employs a Contrastive Contextual Refinement strategy, which synthesizes *intrinsic* implementation details with *extrinsic* differentiation signals.

The refinement process operates in two phases for each tool s_i .

Phase 1: Intrinsic Code Tracing. The agent first utilizes the "jump-to-definition" capability provided by $\mathcal{P}(Repo)$ to perform white-box analysis. It navigates to the declaration site and follows the corresponding paths in $Tree_{clean}(Repo)$ to resolve the tool's actual implementation. By executing a control-flow trace, the agent analyzes how inputs are transformed, which external APIs are invoked, and how return values are constructed. This allows the agent to ground the schema in concrete logic, refining $InputSchema'_i$ and $Return'_i$ with behavior-guaranteed descriptions. For instance, it can refine a generic query: `string` into a specific "SQL statement to execute against the PostgreSQL backend," transforming vague types into semantic specifications.

Phase 2: Extrinsic Contrastive Differentiation. After establishing the intrinsic semantics, the agent validates the description against the server's global context, namely the progressively updated refined tool list $\mathcal{D}(Repo)$, to resolve semantic collisions. In many MCP servers, multiple tools share overlapping functionalities (e.g., different modes of data retrieval). The agent compares the current tool's trace against the generated summaries of sibling tools within $\mathcal{D}(Repo)$. If a high semantic overlap is detected, the agent explicitly triggers a differentiation step: it injects discriminative keywords derived from the implementation differences (e.g., clarifying that `quick_search` queries a "local Redis cache" for low latency, while `deep_search` performs a "full table scan" on the primary database).

This contrastive approach ensures that the final output $\mathcal{D}(Repo)$ is not only technically accurate but also structurally discriminative, maximizing the retrieval system's ability to select the precise tool for a given constraint.

4.4 Stage 3: Server Document Refinement

Input and Output Input: Refined tool documents $\mathcal{D}(Repo)$. Output: Server overview $Doc_{overview}$ and indexable chunks.

A semantic gap often exists between user requirements and server documents: users typically search for general capabilities (e.g., "Manage Notion pages"), while servers expose specific granular functions (e.g., `notion_create_page`). To bridge this gap more effectively, the *Server Document Refiner* synthesizes tool-level evidence into a coherent, retrieval-friendly server-level overview.

The agent reads all refined tool documents $\mathcal{D}(Repo)$ to understand the server's holistic capabilities. It then generates a $Doc_{overview}$ that summarizes the server's application scenarios and explicitly enumerates the functionality available through its toolset. To support retrieval at both the broad intent level and the specific execution level, we construct the final retrieval units using a concatenation strategy. For each tool i , we generate an indexable chunk:

$$Chunk_i = Doc_{overview} \oplus Doc_{tool_i} \quad (6)$$

This hybrid chunk structure preserves the global server context while providing more specific signals for matching fine-grained functional constraints.

5 MCP Retrieval Benchmark

To evaluate MCP server retrieval in realistic open-world settings, we construct the **MCP-Focus Benchmark** to address key shortcomings of existing MCP benchmarks for server retrieval. In particular, many prior benchmarks emphasize tool execution under a known environment or rely on small and homogeneous server collections, which under-represents the scale, heterogeneity, and query ambiguity encountered in real-world retrieval.

We collect 3,763 real-world open-source MCP servers from Glama in total. This corpus reflects the in-the-wild MCP ecosystem and exhibits substantial heterogeneity in domain coverage, code structure, and documentation quality, which is critical for retrieval under noise and variance typical of open-source software.

To approximate real user needs while keeping labels reliable, we adopt a semi-automated query construction pipeline based on MCPEval [16] and adapt it to retrieval with additional human validation. Concretely, we start from Glama-validated tool schemas as ground truth, use an LLM to draft candidate queries grounded in functional signatures, and explicitly encourage paraphrasing and implicit intents to avoid overly templated formulations. For multi-function queries, we further constrain generation to require coherent, scenario-plausible compositions of capabilities, rather than arbitrary concatenations of unrelated requirements. Because retrieval relevance cannot be reliably validated via automated execution feedback, we then apply human-in-the-loop review on sampled (query, server) pairs to filter hallucinated requirements and ensure that each query is plausibly satisfied by the target server's exposed capabilities. We further categorize queries along three dimensions to diagnose performance across difficulty levels:

- **Functional Volume:** stratifies queries by the number of distinct tools required (1, 2, or 3), testing multi-step intents.
- **Semantic Ambiguity:** distinguishes *explicit* queries that are semantically clear and may directly include tool-related keywords from *implicit-intent* queries produced via ambiguity injection (e.g., paraphrasing and implicit intent), evaluating generalization beyond surface cues.
- **Constraint Specificity:** separates queries with explicit technical constraints (e.g., input format) from general requests, probing sensitivity to fine-grained interface details.

Using the above pipeline, we construct a test set of 1,000 queries for each of the 1-, 2-, and 3-function-point settings.²

²The benchmark data and its construction pipeline are provided in our code repository.

Table 2: Main Results: Retrieval performance on MCP-Focus Benchmark for single-server queries targeting one to three tool-level function points. Metrics include $r@5$, $r@10$, $n@5$, and $n@10$; retrievers include BM25, Contriever, co-Condenser, TAS-B, ANCE, and BGE.

Retriever	Doc Type	1-func query				2-func query				3-func query			
		$r@5$	$r@10$	$n@5$	$n@10$	$r@5$	$r@10$	$n@5$	$n@10$	$r@5$	$r@10$	$n@5$	$n@10$
BM25[28]	README	0.422	0.479	0.341	0.359	0.377	0.465	0.315	0.343	0.466	0.536	0.369	0.391
	MCP-Focus	0.668	0.728	0.580	0.600	0.683	0.756	0.576	0.600	0.724	0.796	0.617	0.641
Contriever[13]	README	0.559	0.641	0.451	0.477	0.473	0.595	0.376	0.415	0.594	0.700	0.469	0.503
	MCP-Focus	0.744	0.822	0.641	0.666	0.754	0.819	0.633	0.654	0.816	0.880	0.709	0.729
co-Condenser[6]	README	0.528	0.627	0.421	0.452	0.469	0.583	0.359	0.396	0.569	0.671	0.449	0.481
	MCP-Focus	0.796	0.870	0.567	0.595	0.697	0.788	0.562	0.591	0.775	0.858	0.651	0.678
TAS-B[10]	README	0.522	0.590	0.419	0.441	0.436	0.548	0.340	0.376	0.546	0.645	0.433	0.466
	MCP-Focus	0.682	0.765	0.585	0.612	0.717	0.782	0.589	0.610	0.741	0.817	0.616	0.641
ANCE[38]	README	0.566	0.675	0.465	0.501	0.511	0.608	0.397	0.429	0.631	0.734	0.505	0.538
	MCP-Focus	0.758	0.835	0.654	0.679	0.758	0.819	0.642	0.661	0.827	0.887	0.715	0.734
BGE[37]	README	0.585	0.689	0.475	0.509	0.570	0.659	0.435	0.464	0.623	0.731	0.489	0.524
	MCP-Focus	0.794	0.859	0.684	0.705	0.784	0.874	0.682	0.712	0.833	0.891	0.721	0.740

6 Experiments

In this section, we evaluate the retrieval effectiveness of MCP-Focus across multiple settings. We detail the experimental configuration, report comprehensive quantitative results, and provide detailed analysis to substantiate the performance improvements and generalization of MCP-Focus.

6.1 Experimental Setup

We design our experiments to verify two core hypotheses: first, that improving document quality yields consistent performance gains regardless of the retrieval algorithm used; and second, MCP-Focus is superior to existing repository summarization methods.

Datasets: We utilize the MCP-Focus Benchmark as our primary evaluation set to assess performance in realistic, open-world scenarios. Additionally, to test generalization capability, we evaluate on two public datasets: *MCP-Bench* [35] (112 queries) and *MCP-Universe* [20] (157 queries).

Integration with Retrievers (Orthogonality Analysis): MCP-Focus is a data-centric document enhancement method and is orthogonal to retriever-side optimizations. We therefore evaluate MCP-Focus by integrating it with existing retrievers. Specifically, we consider 1 sparse retriever, BM25 [28], and 5 dense retrievers, Contriever [13], co-Condenser [6], TAS-B [10], ANCE [38], and BGE [37]. For each retriever, we compare two indexing settings: (i) indexing original README files and (ii) indexing MCP-Focus-generated documents. For dense retrievers, we fine-tune models on the corresponding document set for a fair comparison; for BGE, we use bge-base-en-v1.5.

Baselines & Comparative Strategy: We compare MCP-Focus against two distinct baselines to validate different aspects of our contribution:

- **README (Standard Baseline):** This represents the current status quo of the open-source ecosystem. We use this baseline across all experiments to quantify the improvement over raw developer-written documentation.
- **RepoAgent [19] (General-Purpose Baseline):** To verify that general-purpose repository summarization is insufficient for the specific needs of MCP retrieval, we compare MCP-Focus against *RepoAgent*, a state-of-the-art agentic framework for code documentation. Since *RepoAgent* currently supports only Python repositories, we construct a controlled **Python-only subset** of our benchmark (1,464 servers) for this specific comparison to isolate the impact of our function-oriented documentation strategy versus *RepoAgent*’s general structure-oriented approach.

Implementation Details: Both MCP-Focus and *RepoAgent* utilize Qwen3-235B-A22B as the underlying LLM backbone. We maintain identical generation configurations and inference settings for both pipelines to ensure that observed performance differences stem strictly from the framework design rather than underlying model capability.

Metrics: We report Recall@K ($r@k$) and NDCG@K ($n@k$) to measure the presence and ranking quality of the correct server in the top results.

6.2 Main Results on MCP-Focus Benchmark

Table 2 reports retrieval performance on MCP-Focus Benchmark, where queries are grouped by the number of function points they involve (1-func, 2-func, and 3-func). Across all retrievers and all query types, indexing MCP-Focus-generated documents consistently yields substantial gains over the README baseline on both recall ($r@5/r@10$) and ranking quality ($n@5/n@10$), indicating that document quality is a decisive factor for MCP server retrieval.

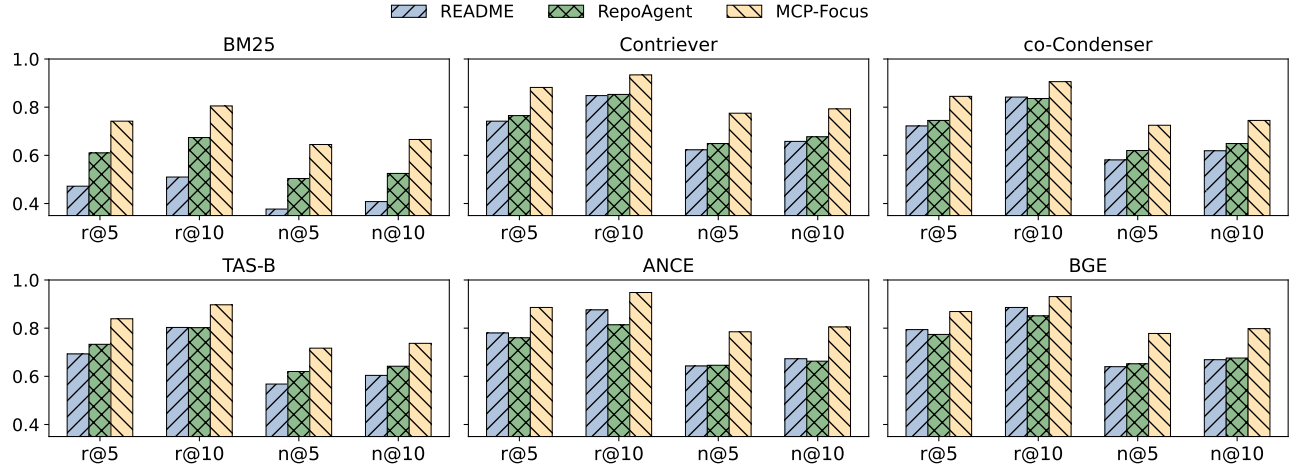


Figure 3: Retrieval results on the Python-based Project subset of MCP-Focus benchmark. Each subfigure corresponds to a retriever (BM25, Contriever, co-Condenser, TAS-B, ANCE, and BGE), comparing README, RepoAgent, and MCP-Focus under $r@5$, $r@10$, $n@5$, and $n@10$.

These improvements are consistent and non-trivial in magnitude across varying query complexities, and they hold for both sparse and dense retrieval models. For example, BM25 improves $r@10$ from 0.479/0.465/0.536 to 0.728/0.756/0.796 and $n@10$ from 0.359/0.343/0.391 to 0.600/0.600/0.641 for 1-func/2-func/3-func queries respectively. Dense retrievers exhibit the same upward trend, e.g., Contriever increases $r@10$ from 0.641/0.595/0.700 to 0.822/0.819/0.880, and BGE achieves the strongest overall performance with $r@10 = 0.859/0.874/0.891$ and $n@10 = 0.705/0.712/0.740$.

Moreover, the performance gap between MCP-Focus and README tends to widen as queries contain more function points. For 3-func

queries, MCP-Focus improves $n@10$ by +0.250 for BM25 (0.391 $\rightarrow$ 0.641), +0.226 for Contriever (0.503 $\rightarrow$ 0.729), and +0.196 for ANCE (0.538 $\rightarrow$ 0.734). This trend suggests that multi-function queries rely more heavily on fine-grained tool semantics and constraints, which are often missing or underspecified in README files but are explicitly captured in MCP-Focus documents through code-grounded, function-oriented descriptions.

6.3 Generalization to Public Benchmarks

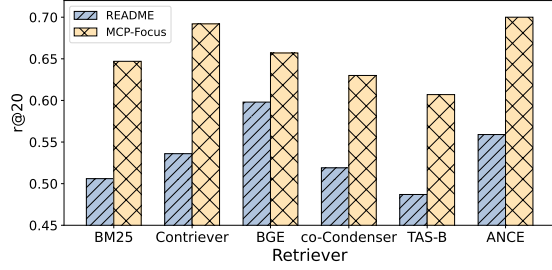
We further evaluate MCP-Focus on two public tool-use benchmarks, *MCP-Bench* and *MCP-Universe*, by adapting their tool-calling instances to MCP server retrieval. As shown in Table 3, indexing MCP-Focus documents consistently outperforms indexing README files across all retrievers, suggesting that our document enhancement strategy generalizes beyond MCP-Focus benchmark.

Results on MCP-Bench. On MCP-Bench, MCP-Focus delivers consistent and substantial improvements for both sparse and dense retrievers. For BM25, $r@10$ improves from 0.554 to 0.848 and $n@10$ from 0.497 to 0.711. Dense retrievers exhibit a similar trend: Contriever and ANCE increase $r@10$ from 0.750 to 0.920, with $n@10$ improving from 0.624 to 0.750 and from 0.589 to 0.737, respectively. These experimental results indicate that MCP-Focus can produce retrieval-oriented functional semantics that align closely with MCP-Bench queries.

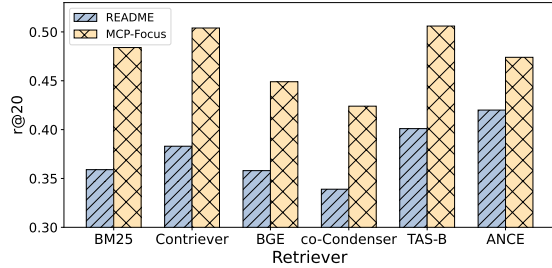
Results on MCP-Universe. On MCP-Universe, overall performance is lower for both README and MCP-Focus, likely because the benchmark is designed for tool invocation and many queries resemble direct tool inputs rather than retrieval-style needs descriptions. Despite this mismatch, MCP-Focus still improves results across all retrievers. For example, BM25 increases $r@10$ from 0.089 to 0.350. Co-Condenser achieves the best overall performance, improving $r@10$ from 0.357 to 0.503 and $n@10$ from 0.219 to 0.286. Overall, the gains suggest that code-grounded document enhancement remains beneficial under distribution shift.

Table 3: Retrieval results evaluated on MCP-Bench and MCP-Universe queries under $r@10$ and $n@10$.

Retriever	Doc Type	MCP-Bench		MCP-Universe	
		$r@10$	$n@10$	$r@10$	$n@10$
BM25[28]	README	0.554	0.497	0.089	0.055
	MCP-Focus	0.848	0.711	0.350	0.172
Contriever[13]	README	0.750	0.624	0.325	0.199
	MCP-Focus	0.920	0.750	0.439	0.263
co-Condenser[6]	README	0.741	0.615	0.357	0.219
	MCP-Focus	0.875	0.714	0.503	0.286
TAS-B[10]	README	0.643	0.530	0.261	0.140
	MCP-Focus	0.839	0.685	0.427	0.262
ANCE[38]	README	0.750	0.589	0.331	0.194
	MCP-Focus	0.920	0.737	0.420	0.260
BGE[37]	README	0.750	0.574	0.376	0.185
	MCP-Focus	0.911	0.725	0.408	0.246



(a) 2-server query retrieval results (r@20)



(b) 3-server query retrieval results (r@20)

Figure 4: Multi-server retrieval results on complex queries under r@20.

6.4 Comparison with Repository-Level Document Method

We compare MCP-Focus against *RepoAgent* [19], a state-of-the-art repository-level document approach for Python projects. As shown in Figure 3, general-purpose code document is insufficient for MCP retrieval. *RepoAgent* primarily targets code readability and maintainability for developers (e.g., explaining modules, classes, and implementation details), whereas MCP-Focus focuses on retrieval-oriented *functional semantics*, i.e., what capabilities a server provides and how its exposed tools can be invoked (tool descriptions, inputs/outputs, and actionable usage cues). This design difference yields a consistent quantitative gap across retrievers and metrics.

MCP-Focus outperforms *RepoAgent* for all evaluated retrievers. For BM25, MCP-Focus improves r@5 from 0.610 to 0.742 and n@5 from 0.504 to 0.645, indicating that our documents expose more retrieval-relevant lexical signals than *RepoAgent*. The gains are also clear for dense retrievers: on Contriever, MCP-Focus increases r@5/r@10 from 0.765/0.853 to 0.882/0.934 and n@5/n@10 from 0.649/0.677 to 0.775/0.793; similar improvements hold for ANCE (r@10: 0.814 $\rightarrow$ 0.948) and BGE (n@10: 0.676 $\rightarrow$ 0.798). Notably, since MCP-Focus and *RepoAgent* use the same underlying LLM to generate documents in our experiments, the improvements are attributable to MCP-Focus’s MCP-specific, bottom-up tool-centric document construction rather than differences in model capacity.

6.5 Complex Query Analysis: Multi-Server Retrieval

Real-world tasks often require composing capabilities from multiple MCP servers. To evaluate such scenarios, we follow a standard

Table 4: Ablation study. *w/o* denotes removing a component from the agentic pipeline: *w/o tool extractor* skips tool extraction and directly generates tool documents; *w/o tool doc refiner* omits tool document refinement and uses the extracted tool schema as tool documents; *w/o server doc refiner* removes the server overview from the final documents.

Methods	MCP-Focus query			
	r@5	r@10	n@5	n@10
BM25[28]	0.649	0.717	0.540	0.563
w/o tool extractor	0.561	0.659	0.456	0.488
w/o tool doc refiner	0.533	0.609	0.427	0.451
w/o server doc refiner	0.570	0.663	0.473	0.504
Contriever[13]	0.746	0.821	0.632	0.657
w/o tool extractor	0.663	0.762	0.543	0.575
w/o tool doc refiner	0.716	0.789	0.589	0.613
w/o server doc refiner	0.677	0.764	0.549	0.577
co-Condenser[6]	0.702	0.794	0.568	0.598
w/o tool extractor	0.623	0.716	0.496	0.526
w/o tool doc refiner	0.647	0.750	0.520	0.554
w/o server doc refiner	0.642	0.743	0.525	0.557
TAS-B[10]	0.677	0.768	0.546	0.576
w/o tool extractor	0.608	0.710	0.488	0.522
w/o tool doc refiner	0.646	0.744	0.520	0.552
w/o server doc refiner	0.620	0.709	0.499	0.528
ANCE[38]	0.764	0.846	0.641	0.668
w/o tool extractor	0.690	0.782	0.569	0.599
w/o tool doc refiner	0.728	0.799	0.602	0.625
w/o server doc refiner	0.690	0.785	0.565	0.595
BGE[37]	0.792	0.856	0.671	0.692
w/o tool extractor	0.701	0.800	0.586	0.619
w/o tool doc refiner	0.740	0.821	0.618	0.644
w/o server doc refiner	0.697	0.800	0.579	0.613

setup [31]: a naive LLM-based decomposer splits each multi-server query into sub-queries by application context, we retrieve candidates for each sub-query, and then fuse ranked lists with Reciprocal Rank Fusion (RRF). We report Recall@20 since multiple servers can be relevant and small cutoffs may under-estimate coverage.

Figure 4 shows that multi-server retrieval remains challenging, with consistent drops from 2-server to 3-server queries across retrievers. Nevertheless, MCP-Focus improves Recall@20 over README for every retriever. For example, on 2-server queries, BM25 improves from 0.506 to 0.647 and ANCE from 0.559 to 0.700; on 3-server queries, BM25 improves from 0.359 to 0.484 and ANCE from 0.401 to 0.506. These gains suggest that retrieval-oriented server documents provide stronger signals for surfacing the correct set of servers under complex, multi-context queries, even with a simple decomposition-and-fusion pipeline.

6.6 Ablation Study

To quantify the contribution of each stage in the MCP-Focus pipeline, we ablate the Tool Extractor, Tool Document Refiner, and Server Document Refiner, and evaluate the resulting documents. Table 4 shows that the full pipeline consistently performs best across metrics and retrievers, indicating complementary benefits for retrieval-oriented document.

Removing the Tool Document Refiner causes a clear drop, showing that structured interface extraction alone cannot capture functional semantics. Using only the extracted tool schema as the document omits retrieval-critical signals (e.g., behavior, intent, and implementation-implied constraints). For BM25, $r@10$ falls from 0.717 to 0.609 and $n@10$ from 0.563 to 0.451; similar trends appear for dense retrievers (ANCE $r@10$: 0.846 $\rightarrow$ 0.799, $n@10$: 0.668 $\rightarrow$ 0.625; BGE $r@10$: 0.856 $\rightarrow$ 0.821, $n@10$: 0.692 $\rightarrow$ 0.644).

The Tool Extractor is also essential. Without explicit extraction, the agent must generate tool documents directly from the repository, increasing the risk of missing MCP-exposed tools amid a large codebase. Performance therefore drops (e.g., Contriever $r@10$: 0.821 $\rightarrow$ 0.762; BGE $r@10$: 0.856 $\rightarrow$ 0.800), suggesting that accurate tool discovery is a prerequisite for reliable function-centric document.

Finally, removing the Server Document Refiner degrades retrieval as well (e.g., ANCE $r@10$: 0.846 $\rightarrow$ 0.785; BGE $r@10$: 0.856 $\rightarrow$ 0.800), indicating that the server overview contributes server-level semantics that improve matching to a server’s overall scope. Overall, the ablations support the joint value of (i) robust tool discovery, (ii) implementation-grounded tool-level refinement, and (iii) server-level synthesis.

7 Conclusion

In this paper, we propose MCP-Focus, a document-centric framework for MCP server retrieval that improves retrieval by enhancing MCP documentation rather than optimizing the retriever. MCP-Focus performs LLM-based white-box analysis of server implementations and uses an MCP-specific bottom-up pipeline to derive tool semantics and synthesize coherent server-level documents. We also build a benchmark of 3,763 real-world open-source MCP servers with queries of varying complexity, and evaluate on it alongside MCP-Bench and MCP-Universe. Results show that MCP-Focus consistently outperforms all baselines across metrics, and ablations validate key components. Future work will improve document generation via LLM post-training.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under the grant number 62595734, and National Key Laboratory of Dataspace Technology and System.

References

- [1] Deepak Bhaskar Acharya, Karthigeyan Kuppan, and B Divya. 2025. Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey. *IEEE Access* (2025).
- [2] Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. 2022. Inpars: Unsupervised dataset generation for information retrieval. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2387–2392.
- [3] Tsai-Tsung Chen, Chuan-Ju Wang, and Ming-Feng Tsai. 2025. Dynamic Margin-based Contrastive Learning for Robust Negative Sampling in Information Retrieval. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2642–2646.
- [4] Zhuyun Dai, Vincent Y Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith Hall, and Ming-Wei Chang. 2023. Promptagator: Few-shot Dense Retrieval From 8 Examples. In *The Eleventh International Conference on Learning Representations*.
- [5] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). *ArXiv abs/2505.02279* (2025).
- [6] Luyu Gao and Jamie Callan. 2022. Unsupervised corpus aware language model pre-training for dense passage retrieval. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. 2843–2853.
- [7] Xuanqi Gao, Siyi Xie, Juan Zhai, Shqing Ma, and Chao Shen. 2025. MCP-RADAR: A Multi-Dimensional Benchmark for Evaluating Tool Use Capabilities in Large Language Models. *ArXiv abs/2505.16700* (2025).
- [8] Hechuan Guo, Yongle Hao, Yue Zhang, Minghui Xu, Peizhuo Lv, Jiezhong Chen, and Xiuzhen Cheng. 2025. A Measurement Study of Model Context Protocol Ecosystem. *arXiv preprint arXiv:2509.25292* (2025).
- [9] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E Hassan. 2025. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers. *arXiv preprint arXiv:2506.13538* (2025).
- [10] Sebastian Hofstätter, Sheng-Chieh Lin, Jheng-Hong Yang, Jimmy Lin, and Allan Hanbury. 2021. Efficiently teaching an effective dense retriever with balanced topic aware sampling. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 113–122.
- [11] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv:2503.23278* [cs.CR]
- [12] Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated Design of Agentic Systems. In *The Thirteenth International Conference on Learning Representations*.
- [13] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118* (2021).
- [14] Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. 2025. NV-Embed: Improved Techniques for Training LLMs as Generalist Embedding Models. In *The Thirteenth International Conference on Learning Representations*.
- [15] Xiaoxi Li, Jiajie Jin, Yujia Zhou, Yuyao Zhang, Peitian Zhang, Yutao Zhu, and Zhicheng Dou. 2025. From matching to generation: A survey on generative information retrieval. *ACM Transactions on Information Systems* 43, 3 (2025), 1–62.
- [16] Zhiwei Liu, Jielin Qiu, Shiyu Wang, Jianguo Zhang, Zuxin Liu, Roshan Ram, Haolin Chen, Weiran Yao, Shelby Heinecke, Silvio Savarese, et al. 2025. Mcpeval: Automatic mcp-based deep evaluation for ai agent models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 373–402.
- [17] Elias Lumer, Anmol Gulati, Vamse Kumar Subbiah, Pradeep Honaganahalli Basavaraju, and James A. Burke. 2025. ScaleMCP: Dynamic and Auto-Synchronizing Model Context Protocol Tools for LLM Agents. *ArXiv abs/2505.06416* (2025).
- [18] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, et al. 2025. Large language model agent: A survey on methodology, applications and challenges. *arXiv:2503.21460* [cs.CL]
- [19] Qinyu Luo, Yining Ye, Shihao Liang, Zhong Zhang, Yujia Qin, Yaxi Lu, Yesai Wu, Xin Cong, Yankai Lin, Yingli Zhang, et al. 2024. RepoAgent: An LLM-Powered Open-Source Framework for Repository-level Code Documentation Generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 436–464.
- [20] Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. 2025. Mcp-universe: Benchmarking large language models with real-world model context protocol servers. *arXiv preprint arXiv:2508.14704* (2025).
- [21] Guozhao Mo, Wenliang Zhong, Jiawei Chen, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. 2025. LiveMCPBench: Can Agents Navigate an Ocean of MCP Tools? *ArXiv abs/2508.01780* (2025).
- [22] Lorenzo Molletta, Giacomo Frisoni, Nicolò Monaldini, and Gianluca Moro. 2025. PORTS: Preference-Optimized Retrievers for Tool Selection with Large Language Models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 10018–10041.
- [23] Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernandez Abrego, Ji Ma, Vincent Zhao, Yi Luan, Keith Hall, Ming-Wei Chang, et al. 2022. Large dual encoders are generalizable retrievers. In *Proceedings of the 2022 Conference on*

- Empirical Methods in Natural Language Processing*. 9844–9855.
- [24] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Forty-second International Conference on Machine Learning*.
 - [25] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*, Vol. 37. 126544–126565.
 - [26] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *The Twelfth International Conference on Learning Representations*.
 - [27] Partha Pratim Ray. 2025. A survey on model context protocol: Architecture, state-of-the-art, challenges and future directions. Authorea preprint.
 - [28] Stephen E. Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Found. Trends Inf. Retr.* 3 (2009), 333–389.
 - [29] Zhengliang Shi, Shen Gao, Lingyong Yan, Yue Feng, Xiuyi Chen, Zhumin Chen, Dawei Yin, Suzan Verberne, and Zhaochun Ren. 2025. Tool Learning in the Wild: Empowering Language Models as Automatic Tool Agents. In *Proceedings of the ACM on Web Conference 2025*. 2222–2237. doi:10.1145/3696410.3714825
 - [30] Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. 2025. Agentic retrieval-augmented generation: A survey on agentic rag. *arXiv preprint arXiv:2501.09136* (2025).
 - [31] Mingyang Song and Mao Zheng. 2024. A Survey of Query Optimization in Large Language Models. *arXiv preprint arXiv:2412.17558* (2024).
 - [32] Kexin Wang, Nandan Thakur, Nils Reimers, and Iryna Gurevych. 2022. GPL: Generative pseudo labeling for unsupervised domain adaptation of dense retrieval. In *Proceedings of the 2022 conference of the North American chapter of the association for computational linguistics: human language technologies*. 2345–2360.
 - [33] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533* (2022).
 - [34] Wenhao Wang, Peizhi Niu, Zhao Xu, Zhaoyu Chen, Jian Du, Yaxin Du, Xianghe Pang, Keduan Huang, Yanfeng Wang, Qiang Yan, and Siheng Chen. 2025. MCP-Flow: Facilitating LLM Agents to Master Real-World, Diverse and Scaling MCP Tools. *ArXiv abs/2510.24284* (2025).
 - [35] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, and Eugene Siow. 2025. MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers. *ArXiv abs/2508.20453* (2025).
 - [36] Zijian Wu, Xiangyan Liu, Xinyuan Zhang, Lingjun Chen, Fanqing Meng, Lingxiao Du, Yiran Zhao, Fanshi Zhang, Yaoqi Ye, Jiawei Wang, Zirui Wang, Jinjie Ni, Yufan Yang, Arvin Xu, and Michael Shieh. 2025. MCPMark: A Benchmark for Stress-Testing Realistic and Comprehensive MCP Use. *ArXiv abs/2509.24002* (2025).
 - [37] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian yun Nie. 2023. C-Pack: Packed Resources For General Chinese Embeddings. *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2023).
 - [38] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate nearest neighbor negative contrastive learning for dense text retrieval. *arXiv preprint arXiv:2007.00808* (2020).
 - [39] Hansi Zeng, Julian Killingback, and Hamed Zamani. 2025. Scaling sparse and dense retrieval in decoder-only llms. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2679–2684.
 - [40] Dawei Zhu, Liang Wang, Nan Yang, Yifan Song, Wenhao Wu, Furu Wei, and Sujian Li. 2024. LongEmbed: Extending Embedding Models for Long Context Retrieval. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 802–816.